1 Current research

1.1 Neuro-Symbolic Action Planning with Qwen–CLIPS

We integrate natural-language interpretation and rule-based action planning through Qwen–CLIPS. The architecture separates the prediction of task goals from their expansion into actions supported by Justina's executor. Its scope is defined by the implemented families, vocabulary, and action repertoire of the General Purpose Service Robot (GPSR) task in RoboCup@Home. Qwen produces an intermediate representation of the requested goals; CLIPS generates the corresponding symbolic plan using explicit production rules.

Domain-Specific Interpretation We fine-tune Qwen2.5-7B using Quantized Low-Rank Adaptation (QLoRA) on a corpus of 40,000 command–goal pairs. The corpus is generated from GPSR task templates and catalogues, extended locally with alternative linguistic formulations. The interpreter returns a JSON object containing an ordered list of canonical goals, such as `go`, `find`, `take`, and `deliver`. Goals specify entities, destinations, types, and supported qualifiers rather than low-level motion commands.

Inference is exposed through an HTTP server that loads the base model and its adapter once. The server normalizes the input, generates the goal representation, and validates it before returning the response. A ROS 2 semantic-parser node in Justina sends the textual command to the server through `/translate`. The server can run on the robot's computer or on a separate GPU-equipped computer, allowing language inference to be separated from onboard perception and control. A remote deployment requires network connectivity to that server, but does not require a third-party language-model API.

From Goals to an Executable Action Vocabulary The planning pipeline follows four stages:

1. **Command interpretation.** A textual instruction, entered directly or obtained from speech recognition, is submitted to Qwen. The response preserves the requested ordering and entities through canonical goals; the predicted interpretation remains subject to semantic evaluation.
2. **Fact construction.** The goal-conversion node parses the JSON and constructs ordered `gpsr-goal` facts. Their slots encode the step, goal name, target, entity kind, destination, relation, value, and supported qualifier. The language model does not generate arbitrary CLIPS code.
3. **Rule-based expansion.** The facts are inserted into CLIPS working memory and planning is triggered. Rules expand goals while tracking expected state, including the robot's location and the held object, to connect navigation, search, acquisition, delivery, and interaction actions.
4. **Plan publication.** The CLIPS node publishes the ordered action sequence through ROS 2 for the supervisor and executor. Each step identifies an action and its arguments. The planning layer specifies the sequence; perception,

navigation, manipulation, and interaction modules perform the corresponding operations.

For example, “Bring me a mug from the cabinet” is represented as:

```
{"goals": ["go(cabinet)",
           "find(mug, kind=object)",
           "take(mug)",
           "deliver(mug, to=me)"]}
```

The search goal is converted into a fact of the form:

```
(gpsr-goal (step 2) (name find)
  (target "mug") (kind object)
  (destination "") (relation "")
  (value "") (qualifier ""))
```

CLIPS expands the sequence into navigation to the cabinet, object search and acquisition, return to the instruction point, approach to the operator, and delivery. Announcements and completion messages are added according to the executor contract. These state transitions describe expected effects of the plan, not observations that physical execution has succeeded.

Interaction and Executor Integration Interaction rules connect person search, visual attention, speech, listening, and stored dialogue information. Visual attention can remain active across consecutive speech and listening actions directed at the same person and is released at the end of the interaction. General announcements do not require person-directed attention. Symbolic arguments and protocol keys must match the executor’s supported values; syntactically valid JSON alone does not ensure a correct or usable plan.

This separation supports inspection of interpretation and planning errors and permits explicit maintenance of the domain rules. Coverage remains bounded by the trained command families, intermediate representation, and executor capabilities. Domain extensions require explicit updates to the supported goal representation, production rules, and executor capabilities.